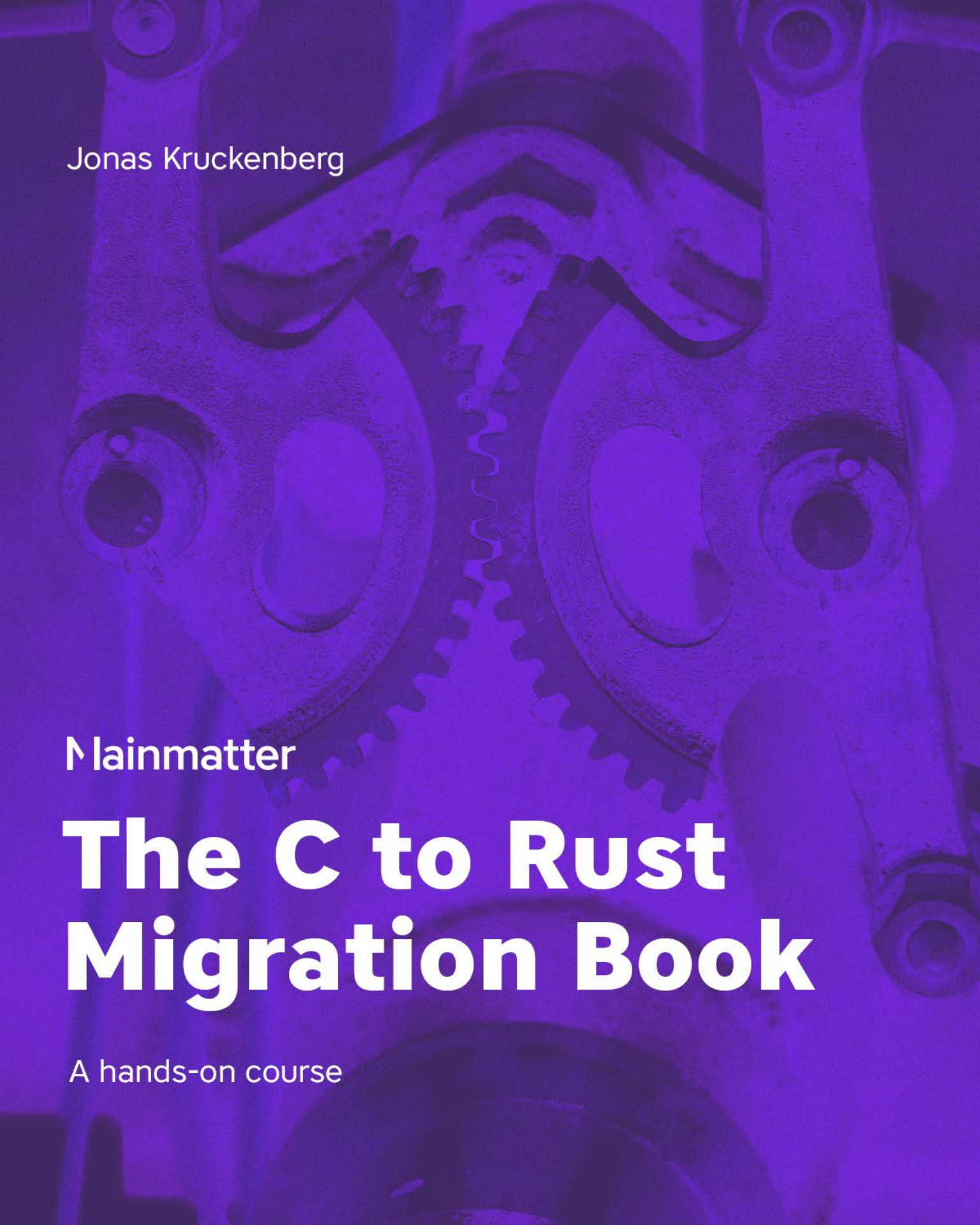




Jonas Kruckenberg

Mainmatter

The C to Rust Migration Book

A hands-on course

The C to Rust Migration Book

A hands-on course by Mainmatter

Jonas Kruckenberg

Artur Jakubiec

Contents

Welcome	1
1 Chapter 1: Basics	5
1.1 What is FFI? Calling C from Rust	5
1.2 When hand-written bindings drift	7
1.3 Generating Rust bindings with bindgen	9
1.4 Exposing Rust to C with cheadergen	10
1.5 FFI-safe types	12
1.6 The FFI boundary as a firewall: validate and narrow	15
1.7 High-quality FFI APIs	17
2 Chapter 2: Intermediate	19
2.1 How to rewrite a module	19
2.2 Allocation, allocators, and mixing them	22
2.3 Using lldb/gdb	26
2.4 Clippy lints for unsafe Rust and static verification	29
2.5 Replacing C-isms with Rust-isms	32
2.6 Dynamic analysis with Miri	36
2.7 Performance benchmarking	39

Welcome

Welcome to the **“C to Rust Migration Book”**!

This course will teach you techniques for migrating C codebases to Rust. You’ll learn how to maintain a safe mixed C-Rust codebase, incrementally migrate modules, translate common C idioms to Rust, and use the debugging tools you’ll need when things inevitably break.

We assume prior knowledge of Rust and some C.

You’ll build up your knowledge in small, manageable steps. By the end of the course, you will have solved many exercises and should be prepared to migrate even larger C codebases to Rust.

Methodology

This course is based on the “learn by doing” principle. It has been designed to be interactive and hands-on.

[Mainmatter](#) developed this course to be delivered in a classroom setting, over four days: each attendee advances through the lessons at their own pace, with an experienced instructor providing guidance, answering questions, and diving deeper into the topics as needed. If you’d like to organize a private session for your company, please [get in touch](#).

You can also take the course on your own, but we recommend you find a friend or a mentor to help you along the way should you get stuck.

Formats

You can go through the course material [in the browser]([\[TODO: insert URL\]](#)) or [download it as a PDF file]([\[TODO: insert URL\]](#)), for offline reading.

Structure

On the left side of the screen, you can see that the course is divided into sections. To verify your understanding, each section is paired with an exercise that you need to solve.

You can find the exercises in the [companion GitHub repository](#).

Before starting the course, make sure to clone the repository to your local machine:

```
git clone https://github.com/mainmatter/migrating-c-to-rust
```

We also recommend you work on a branch, so you can easily track your progress and pull in updates from the main repository, if needed:

```
cd migrating-c-to-rust
git checkout -b my-solutions
```

All exercises are located in the `exercises` folder. Each exercise is structured as a Rust package. The package contains the exercise itself, instructions on what to do (in `src/lib.rs`), and a test suite to automatically verify your solution.

Tools

To work through this course, you'll need:

- **Rust**. If `rustup` is already installed on your system, run `rustup update` (or another appropriate command depending on how you installed Rust on your system) to ensure you're running on the latest stable version.
- *(Optional but recommended)* An IDE with Rust autocompletion support. We recommend one of the following:
 - [RustRover](#);
 - [Visual Studio Code](#) with the [rust-analyzer](#) extension.
- A C compiler. The one provided by your operating system will be good enough.

Workshop runner

To verify your solutions, we've also provided a tool to guide you through the course: the `wr` CLI, short for "workshop runner". Install `wr` by following the instructions on [its website](#).

Once you have `wr` installed, open a new terminal and navigate to the top-level folder of the repository. Run the `wr` command to start the course:

```
wr
```

`wr` will verify the solution to the current exercise.

Don't move on to the next section until you've solved the exercise for the current one.

We recommend committing your solutions to Git as you progress through the course, so you can easily track your progress and "restart" from a known point if needed.

Enjoy the course!

Authors

This course was written by [Jonas Kruckenberg](#), Engineering Consultant at [Mainmatter](#), and Artur Jakubiec.

Jonas Kruckenberg is a systems engineer working on operating systems and low-level infrastructure. He is the lead author of k23, an experimental high-reliability operating system. As a TC39 Invited Expert, he helps shape the future of the web by bringing non-browser perspectives to JavaScript language standardization.

Artur Jakubiec is a long-time C/C++ engineer and was one of the main developers of Cubasis. He is now a tech lead at a company building enterprise SaaS productivity software, currently migrating it from C++ to Rust.

Chapter 1

Chapter 1: Basics

In this chapter we'll cover the FFI foundations: what FFI is, how we use it to interoperate between C and Rust, and the common pitfalls.

Exercises

The exercises for this section are located in `exercises/01_intro`

Exercise

The exercise for this section is located in `01_intro/00_README` `\newpage{=latex}`

1.1 What is FFI? Calling C from Rust

Programming language design has taken many different paths over the years. We have C-like languages that are compiled all the way down to machine code, but we also have interpreted languages, lazy languages, languages that compile to a portable bytecode, garbage-collected languages, and so much more.

As software engineers, we do not want to duplicate work unnecessarily. So what happens when work exists in a language other than the one we're using?

When developing in Python, for example, we don't want to port all the code we need just to use it. Instead, we rely on infrastructure that lets us call functions and use types from other programming languages.

This infrastructure, which bridges different type systems (or lack of type systems), execution models, and code-organization concepts, is called a *Foreign Function Interface* (abbreviated *FFI* from here on out). Most modern languages have some way to interoperate with others through FFI. The exact syntax and options vary by language, but they almost always have one thing in common: they represent functions as if they were C functions.

This is called the “C ABI” (C Application Binary Interface) or “C calling convention” and, as the name suggests, is a convention on how to call functions (which CPU registers hold what arguments, which register(s) hold return values, when to spill to the stack, etc.)¹. Notice that this is just a convention that the industry agreed on over time. The C ABI is predictable, simple, and stable, and it has been so for decades, which is how it became the de facto standard for language interoperability.

Calling an FFI function

Let’s say for example we have a Rust program that needs to call the `time` function from `libc` (a C static library)². We would use the following construct:

```
type time_t = i64;
unsafe extern "C" {
    fn time(time: *mut time_t) -> time_t;
}
```

Before we dissect the syntax though: you may have already noticed that nowhere in this snippet do we ask for `libc`. Why is that?

This is because object file formats (ELF on Linux, Mach-O on macOS, PE on Windows) are all old and therefore simplistic. They have *one global namespace*, called a *symbol table*, that all functions (and statics) share. So you cannot say “call `time` from `libc`”, you can only say “call a function named `time`”.

When a compiler builds a program, all function calls reference the function *by symbol* (“call function named X”). To make this actually executable, we need to replace every symbol reference with the actual address of the function. This happens *after* compilation during the *linking* step, where a separate program, the *linker*, aggregates all object files that make up your final program, lays them out on disk, and then resolves these symbol names.

To call `time` from `libc` we therefore need Rust to emit a reference to the `time` symbol and make sure the `libc` file is also passed to the linker.

This solves the problem of *what* to call, but we still need to figure out *how* to call that function: How many parameters does the function accept? What are the types of those parameters? How many values does it return? Remember the symbol references above are plain string names. They carry no information about the function’s argument or return types, so `rustc` has no way to work them out on its own.

This is why we need to tell it about `time`’s signature through a so-called “extern block” (or sometimes an “extern C block”) above. This block *declares* items that are not *defined* in the current crate. Each item we declare is a promise to the compiler: “this is the correct signature of this symbol, trust me”. We commonly refer to it as a **binding**.

¹Technically, there is no “single” calling convention. Every architecture defines its own “C Calling Convention”. For example, the RISC-V C Calling Convention is defined [here](#) and lays out the sizes of C primitives and how arguments and return values are passed from and to functions. x86 architectures have *many* different calling conventions: the Microsoft x64 calling convention and the System V ABI are the most common, but many calling conventions exist to improve calling performance (fastcall, regcall, and more). When we say “the C calling convention” we usually mean “the C calling convention commonly used on this OS+architecture combination”.

²Yes, generally speaking `libc` is distributed not as a static but as a *dynamic* library, which is a completely different way of linking and calling functions.

Get the signature wrong and your Rust program will pass garbage to the FFI function without any way to check this at compile-time. The exact implementation won't be known until link-time, much later than the compiler's type-checking pass. This is why bindings are marked `unsafe`: you as the programmer have to ensure signatures are correct.

This is the foundation of all Foreign Function Interfaces in Rust. In later exercises we will see how to make this much safer and more ergonomic.

Head to the exercise

You'll write this block for a `bm_add` function implemented in C.

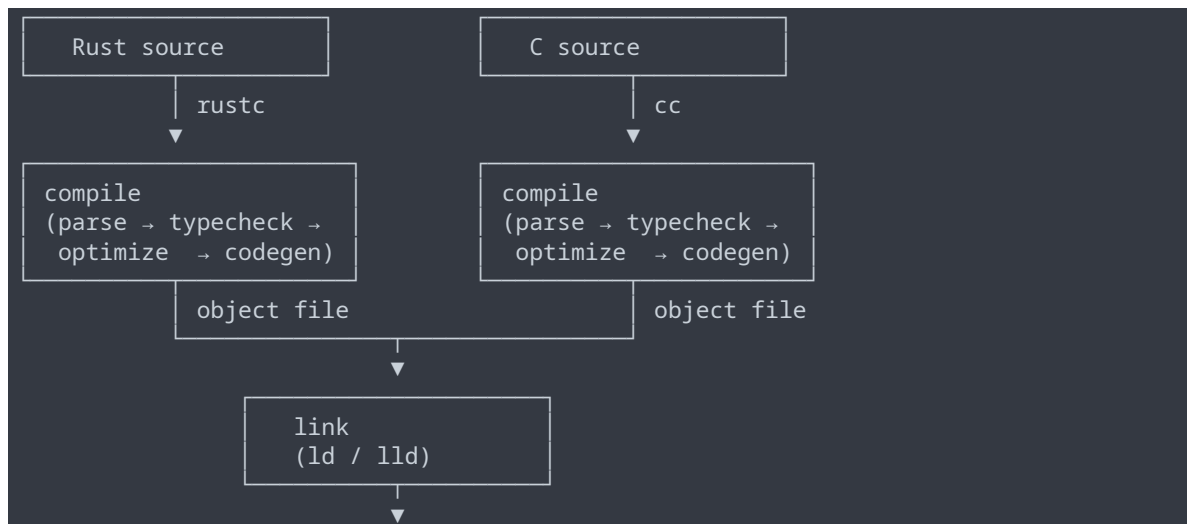
Exercise

The exercise for this section is located in [01_intro/01_what_is_ffi](#) \newpage{=latex}

1.2 When hand-written bindings drift

In the previous exercise you wrote an `extern "C"` block binding to C code. It worked great, but can you imagine doing that for *hundreds*, maybe *thousands*, of functions and types? There's a subtler problem, too. Hand-written bindings will *inevitably* drift out of sync with the C header, especially at scale.

Neither the C nor the Rust compiler can detect this, because each operates in its own small universe, called a *compilation unit*. A compilation unit is the single chunk of work that flows through the various stages of the compiler. In C/C++ this would typically be a `.c / .cpp` file and in Rust that is typically a single crate. The compiler parses, typechecks, and optimizes the compilation unit, then produces an *intermediate* object file. Once all the compilation units that make up your project are built, the intermediate object files are gathered and passed to the *linker*, which produces the final executable or library.



```
final binary
```

This model is great for compilation performance, because we can process many of these compilation units *in parallel*. There is a catch: they *must not* share information, since that would destroy the parallelism. Each compilation *must* be its own self-contained universe.

And even if we had a mechanism to share information between compilation units, we would need to make it language-agnostic so that a C compilation unit and a Rust compilation unit can interoperate. How would that even work with wildly different type systems?³

So compilers have resorted to *manual escape hatches* like the `extern "C"` block you wrote. You as the programmer promise to the compiler that a function with a given name and a given signature will exist at link time, and the compiler takes your word for it.

Possible consequences

The consequences of drift between the two versions can be dire: if C expects `int` but Rust expects `float`, the integer bit patterns are reinterpreted as floats. The result is almost always nonsensical.

It can be worse: passing too many or too few arguments can result in either overwriting important information on the stack or reading garbage from it.

Here is an especially insidious case. The function, as we have established, is the following:

```
int bm_add(int a, int b);
```

but now in the Rust code, we expect it to return 64-bit integers instead of 32-bit integers:

```
use std::ffi::c_longlong;
unsafe extern "C" {
    fn bm_add(a: c_longlong, b: c_longlong) -> c_longlong;
}
```

How will this example fail? If you try it yourself, you will notice that it doesn't. Function arguments are passed in registers, and on a 64-bit machine those registers are 64 bits wide, so a 32-bit `int` occupies one of them with room to spare. What the upper half of the register holds is not something the declaration can rely on: some ABIs require a narrow integer to be sign-extended across the full register, others leave the high bits unspecified entirely. Nothing widens the argument on your behalf. Reading it back as a 64-bit integer just happens to give the right answer here.

You can see this in Compiler Explorer here: [The correct version](#) passes `a` and `b` in the `a0` and `a1` registers (the `li a0, 1` and `li a1, 2` lines do the loading) and passes the return value in the `a0`

³Yes, the LLVM bitcode embedded by toolchains for LTO (Rust's `-Clto=thin` `-Cembed-bitcode=yes` and Clang's `-flto=thin`) *does* carry the information to catch problems like this at link-time and it *is* cross-language, but linkers do not generally validate it. (The `wasm-ld` linker does, but only for Wasm.) The reasons are many, the most important being that validating it would break existing compiler optimizations. You *could* write custom LLVM-bitcode parsing tooling to check this if you wanted.

register. Those registers are 64 bits wide, so you can see the [incorrect c_ulonglong-expecting version just works](#): the values are small and positive, and nothing has put anything else in the upper halves of a0 and a1.

Now, if your 64-bit numbers stay below the 32-bit max value, everything just happens to work. But it is *very* fragile. As you can see here, [when we compile to a 32-bit target instead](#), everything breaks and we end up adding 0 to a instead.

This example is particularly scary, because for 99% of inputs and deployment configurations the mistake is virtually consequence-free. The addition will continue to work as expected. But as soon as the input is unusual, the deployment target is different, *or you add code that will make the compiler change the generated code even a bit*, this will be a bug that takes you weeks to troubleshoot in the worst case.

Head to the exercise

The exercise turns this around: the Rust binding stays fixed and correct-looking while the C implementation drifts underneath it. It is the same mismatch, and therefore the same bug, from either side. Drifting the C side just keeps the Rust crate compiling, so you can run it and watch what happens.

Play with the different “drifted” C implementations of `bm_add` to see what happens on your machine. Feel free to also play around a bit with the Compiler Explorer playgrounds to see how different miscompilations manifest in the generated assembly.

Exercise

The exercise for this section is located in [01_intro/02_drift](#) \newpage{=latex}

1.3 Generating Rust bindings with bindgen

In the previous exercise you saw how easy it is for manual bindings to go out of sync, and the scary silent corruption that causes. This section introduces a tool that avoids all of this: [bindgen](#). It’s a library you call from your Rust build script to generate Rust bindings to C code from C header files. It runs libclang on a C header and emits matching Rust declarations: `extern` blocks, `#[repr(C)]` structs, integer constants. The header remains the single source of truth, with the Rust bindings being generated on every build.

You call `bindgen` from your build script like so:

```
let bindings = bindgen::Builder::default()
    .header("c_src/bm_legacy.h")
    // tell Cargo to re-run when headers change
    .parse_callbacks(Box::new(bindgen::CargoCallbacks::new()))
    .generate()
    .expect("bindgen failed");

let out_path = PathBuf::from(env::var("OUT_DIR").unwrap());
bindings.write_to_file(out_path.join("bindings.rs")).unwrap();
```

This writes the generated file into the crate's `OUT_DIR` (usually `target/<debug or release>/build/<crate name and hash>/out`). Inspecting the generated file, you will see something like this:

```
/* automatically generated by rust-bindgen 0.72.1 */

unsafe extern "C" {
    pub fn bm_add(a: ::std::os::raw::c_int, b: ::std::os::raw::c_int) -> ::std::os::raw::c_int;
}
```

You can then pull this generated file into your crate with the `include!` macro like so:

```
mod sys {
    include!(concat!(env!("OUT_DIR"), "/bindings.rs"));
}
```

This is great: we've gained end-to-end type checking. A change in the C header will not silently corrupt our Rust code. But remember, these bindings are as unsafe as the C code itself. It's your responsibility to use them correctly.

You would typically wrap them in a safe, idiomatic API. This is the common `-sys` crate pattern: a `<crate>-sys` crate holds the raw bindings, while the `<crate>` crate on top exposes the safe abstractions.

Head to the exercise

Replace the hand-written `extern` block from the previous section with `bindgen` in `build.rs`. The tests stay identical.

Tip

`cargo expand -p bm_bindgen` shows what `bindgen` produced. Requires [cargo-expand](#).

Exercise

The exercise for this section is located in [01_intro/03_bindgen](#) \newpage{=latex}

1.4 Exposing Rust to C with `cheadergen`

So far we've always called C from Rust. Now we go the other way: our goal is to *migrate* C code to Rust, so eventually C will need to call into our new Rust code.

To make a Rust function callable from C we'll need to first annotate it correctly:

```
use std::ffi::c_char;

#[unsafe(no_mangle)]
pub extern "C" fn bm_strlower(s: *mut c_char) {
    // ...
}
```

Much like in the FFI function declaration we saw in the first exercise, we instruct `rustc` to use a C-compatible calling convention through `extern "C" fn` (without it, Rust uses its own ABI, which of course C knows nothing about).

`#[unsafe(no_mangle)]` keeps the symbol name verbatim in the compiled object. Otherwise Rust would emit a so-called *mangled* symbol name, a pseudo-random identifier that in regular Rust code prevents two identically named types or functions from causing “duplicate symbol” linking errors.

In this case we do want the symbol name to be the same plain identifier we gave it, so the C code linking to us can actually find the function.

The second piece is the header file that allows the C compiler to understand the API. You could write it by hand, but you’d be maintaining two sources of truth. [cheadergen](#) (the inverse of [bindgen](#)) generates C headers from Rust code.

`cheadergen`, created and maintained by our colleague Luca Palmieri, reads your Rust code and generates a C header for the relevant FFI-visible symbols.

Install the CLI from the [cheadergen releases page](#).

Then you can run the following command in your crate root (for example, the current exercise crate):

```
cheadergen generate
```

and a header will appear:

```
/* Generated by cheadergen – do not edit */  
  
void bm_strlower(char *s);
```

`cheadergen` comes with two ways to configure its output. The first is a project-wide `cheadergen.toml` file. Here you can tweak the style of the generated headers, include custom preamble text, and more. See the full documentation [here](#).

The second configuration option is the `#[cheadergen::config(...)]` attribute. You can apply this attribute to any Rust type. It lets you configure per-type options such as renaming fields, skipping exporting, forcing exporting, and more. See the full documentation [here](#).

A note on `cbindgen`

If you’ve been researching Rust FFI tooling, you’ve undoubtedly come across [cbindgen](#), a similar tool maintained by Mozilla, and you may be wondering: “Why did you just introduce me to this custom tool instead?”

Well, we wouldn’t have built a new tool if the existing one fit. `cbindgen` dates back to 2017, when the Rust ecosystem looked quite different. To read your Rust and emit headers, it had to parse the source itself with a custom syn-based parser. At the time, that was the only way, since the compiler didn’t expose enough information.

But the technique has real limits. The custom parser doesn’t always agree with `rustc`, it often needs manual steering and fixups, and when it hits something it can’t handle it tends to silently

skip generating a header rather than tell you why. None of this is a knock on `cbindgen`: it's what the constraints of the era allowed. `cheadergen` exists because those constraints have since eased, letting us build something with better diagnostics and fewer surprises.

Head to the exercise

You'll port the `bm_strlower` function to Rust and call `cheadergen generate --lang c --output-dir c_test -p bm_cheadergen` to generate the header automatically.

Exercise

The exercise for this section is located in [01_intro/04_cheadergen](#) \newpage{=latex}

1.5 FFI-safe types

We have passed data from C to Rust and from Rust to C in the previous exercises. In doing so we have mostly constrained ourselves to pointers or `c_`-prefixed primitives. At this point you may wonder: "What if I want to pass a `struct`, or a `String`?"

Primitives

All primitives (`i8..i64`, `u8..u64`, `f32`, `f64`, `bool`, and pointers) are always safe to share across an FFI boundary, since they correspond 1:1 with C types. `Option<NonNull<T>>` deserves special mention because it is *also* FFI-safe and guaranteed to have the same representation as `*mut T`.

Note that C's `int` type and Rust's `i32`, for example, *almost always* mean the same thing. But because the C standard's definition of these types is loose, there exist architectures for which this is not true. This is rare enough that the Rust team decided Rust's integer types are FFI-safe. If you want to be sure, though, you can use the `std::ffi::c_*` types such as `c_int` or `c_longlong`.

`repr(C)`

Much like we need to use the C calling convention to make our Rust functions interoperable, we need to use the C struct layout to make our structs interoperable. This is because (just like functions) Rust reserves the right to change the struct layout at any time (it is "undefined")⁴. To have a stable layout that other languages can understand, we need to mark our structs with the `repr(C)` attribute.

⁴The compiler does this to be smart and optimize things. For example, [here](#) is the code snippet you saw above in Compiler Explorer again. If you look at the rightmost "Compiler Output" pane you will see the actual layout of each struct. You can see that for `Bar` the compiler reordered the fields to remove the interior padding byte.

```
// we know the layout of this struct is always (16 bytes in total, because the
// `usize` gives the whole struct an alignment of 8):
// - `a` - 8 bytes
// - `b` - 1 byte
// - 1 byte padding, so that `c` is 2-byte aligned
// - `c` - 2 bytes
// - 4 bytes trailing padding, so that the size is a multiple of the alignment
#[repr(C)]
struct Foo {
    a: usize,
    b: u8,
    c: u16,
}

// we DO NOT know the layout of this struct!
struct Bar {
    a: usize,
    b: u8,
    c: u16,
}
```

`repr(C)` also applies to *tuple structs* where the layout is exactly the same except the fields don't have names.

The `repr(..)` attribute can also be used on enums:

```
// This corresponds to named u8 constants, where A = 0, B = 1, C = 2
#[repr(u8)]
enum Foo {
    A,
    B,
    C,
}

mod explicit_tags {
    // You can of course also assign explicit tags
    #[repr(u8)]
    enum Foo {
        A = 5,
        B = 2,
        C = 8,
    }
}

mod c_abi {
    // repr(C) also works and uses the "default enum size and sign for the target platform's C ABI"
    #[repr(C)]
    enum Foo {
        A,
        B,
        C,
    }
}
```

You can use enums with fields even though they don't have an inherent C equivalent. Rust

defines a stable mapping [here](#).

```
// A definition like this...
#[repr(u8)]
enum TwoCases {
    A(u8, u16),
    B(u16),
}

//...is in essence just syntax sugar for this:
union TwoCasesRepr {
    A: TwoCasesVariantA,
    B: TwoCasesVariantB,
}

#[repr(u8)]
#[derive(Clone, Copy)]
enum TwoCasesTag {
    A,
    B,
}

#[repr(C)]
#[derive(Clone, Copy)] // union fields have to be Copy
struct TwoCasesVariantA(TwoCasesTag, u8, u16);

#[repr(C)]
#[derive(Clone, Copy)]
struct TwoCasesVariantB(TwoCasesTag, u16);
```

So, essentially, an enum with fields decomposes into a tag enum and a union of its fields.

repr(transparent)

`repr(transparent)` doesn't appear as often but warrants special mention. It is an attribute that can only be used on types with a single sized field. It guarantees that the layout of the outer type will be exactly the same as that of the inner type.

```
// Foo is guaranteed to have the same representation as `*const u8`!
#[repr(transparent)]
struct Foo(*const u8);

mod generic {
    use std::marker::PhantomData;
    // because this is concerned with _sized_ fields (fields that have a size)
    // fields that have no size such as PhantomData can still be used!
    #[repr(transparent)]
    struct Foo<T>(*const u8, PhantomData<T>);
}
```

`repr(transparent)` comes in handy if you need to cast pointers or transmute between types.

Types that cannot cross the FFI boundary

The list is long, but as a rule of thumb, two kinds of type cannot be shared: anything generic, and any Rust type that is not explicitly FFI-safe, such as `String`.

Generics are not FFI-safe because the compiler will monomorphize a concrete version of the struct for each type passed into the generic. If we pass the type across the FFI boundary, the C compiler, which does not know about monomorphization, cannot know which version to pick. There exists no ABI that represents generics.

Head to the exercise

There you will find an FFI function that attempts to pass types that are not FFI-safe. Notice the compiler-generated warnings: it is your job to fix them by using FFI-safe types.

Exercise

The exercise for this section is located in [01_intro/05_ffi_safe_types](#) \newpage{=latex}

1.6 The FFI boundary as a firewall: validate and narrow

By now you've written `extern "C"` functions in both directions, seen `bindgen` generate Rust code and `cdeclgen` generate C headers.

On paper that is everything you need, and it's tempting to go ahead and rewrite your project now. Take your C API and translate it one-for-one into Rust.

The problem is that often you can't, and just as often you shouldn't. Real-world C is complicated, idiosyncratic, and (let's admit it!) full of skeletons. You're likely thinking about a Rust rewrite not just to address security problems or performance issues; rewriting in Rust is a chance to clean up your codebase: re-establish module boundaries, drop legacy assumptions, write the module you wished you had.

At the same time big-bang rewrites (where you replace the entire codebase at once) famously never work. Which leaves a problem: you have a clean Rust codebase, a messy legacy codebase, and a need to keep both working together for *some time*⁵.

During this transitional period, the FFI layer is where those worlds meet. It has to bridge unsafe-everything-goes C and the borrow checker, and bridge your new design and the old one.

Many people think FFI is about *type translation*. That's true as far as it goes, but it's like saying programming is about typing words into a computer: it misses the point. The *main* job of your FFI interface is **establishing confidence**. In a legacy codebase you rarely know with 100% certainty that reality matches your assumptions: some caller might actually pass a null pointer, or the `*const c_char` that used to be UTF-8 might now hold binary data.

⁵The exact amount of time of course varies with the scale of your codebase, but from hard-won experience: measure this in years. C codebase migrations have a funny habit of taking much, much longer than you think. As a rule of thumb: take your worst-case estimate, double it, and add a year. Really.

The FFI boundary is your chance to turn legacy uncertainty into known-good state before it crosses into your new system. Uncertainty leaks. Your FFI must be a vigilant firewall against it.

To this end, we’ve settled on a handful of rules that we now live by.

Always validate your inputs

Validate every assumption at the FFI boundary. Null checks, length checks, UTF-8 checks. Panic or return an error if anything is even slightly off. In web servers, the HTTP handler function is your primary interface to the “chaos world” of the internet. Think of FFI functions the same way: they are your primary interface to the legacy-C “chaos world”.

You should also lean on Rust’s type system so “forgot to check” isn’t even an option. For example: at Mainmatter, we encourage contributors to use `Option<NonNull<T>>` instead of `*mut T` as much as possible, for the simple reason that a `*mut T` can be dereferenced directly (`*ptr`), which triggers UB if the pointer is null. You would have to add a manual `if ptr.is_null() {}` check before every pointer dereference. With `Option<NonNull<T>>`, on the other hand, the type system *forces* you to handle the `None` case explicitly. Even the laziest `.unwrap()` will result in a loud panic instead of potentially silent UB.

```
// `*mut T` silently accepts null. You'll remember to check. Until you don't.
pub extern "C" fn bm_do_thing(input: *mut Thing) -> BmResult { /* ... */
}

// `Option<NonNull<T>>` encodes null as `None` – the compiler forces the branch.
pub extern "C" fn bm_do_thing(input: Option<NonNull<Thing>>) -> BmResult {
    let Some(input) = input else {
        return BmResult::ErrInvalid;
    };
    // `input: NonNull<Thing>`, statically non-null.
    /* ... */
}
```

Don’t overload primitives

Legacy C code has a habit of packing several meanings into one return type. POSIX `read()` is a good example: `-1` means error (check `errno`), `0` means end-of-file, anything positive is the byte count. It is *very easy* to mess this up. Rust gives you better tools: `bool` for yes/no, an enum for branching outcomes, an out-parameter for the count. These types usually translate well into C headers too, so use them.

```
#[repr(C)]
pub enum BmReadStatus { Ok, Eof, IoError }

pub extern "C" fn bm_read(/* ... */, out_bytes: Option<NonNull<usize>>) -> BmReadStatus;
```

Head to the exercise

We'll continue the porting work by looking at `bm_normalize_url`, in `exercises/_bm/src/normalize.c`. It will normalize a given URL by lowercasing it and writing the normalized string into the provided buffer. The exercise already contains our *"first draft"* Rust translation: a naive transliteration from C to Rust that has a number of issues.

Let's apply what we learned above and fix the implementation.

Hint: check the exercise's tests for the expected behavior of `bm_normalize_url`.

Exercise

The exercise for this section is located in `01_intro/06_validate_and_narrow` \newpage{=latex}

1.7 High-quality FFI APIs

We previously introduced two "mechanical" rules: always validate your inputs, and don't overload primitives. The two that follow are more conceptual. Don't let that fool you: they are just as important as the first two, if not more so.

Documentation, documentation, documentation

Some invariants you can check at runtime. Many you can't: who owns this pointer, whether the string is copied or borrowed, whether `free` has already run, what each error variant means. Write them all down.⁶ Rust's `# Safety` convention, being just a comment, works with extern "C" functions too, and `changelog` emits those sections as C doc comments in the header files:

```
/// Normalize a URL into the caller's buffer.
///
/// # Ownership
/// `url` and `out` are borrowed for the call; the caller continues to own both.
///
/// # Safety
/// 1. `url`, if non-null, must be a [valid] pointer to a NUL-terminated UTF-8 byte sequence.
/// 2. `out`, must be a [valid], non-null pointer to a writable buffer of at least `out_len` bytes.
///
/// # Errors
/// - `BmResult::ErrInvalidUrl` if `url` is null or not valid UTF-8.
/// - `BmResult::ErrBufferTooSmall` if the result wouldn't fit in `out_len` bytes.
///
/// [valid]: https://doc.rust-lang.org/std/ptr/index.html#safety
#[no_mangle]
pub unsafe extern "C" fn bm_normalize_url(
```

⁶You may say, jadedly, that no one ever reads comments, and you may be right. But with the rise of LLMs, *something* does read them. We've found that LLMs struggle with the nuanced, unspoken invariants of FFI code, which is not surprising. Turning as many of these unspoken invariants into spoken ones helps you get better mileage out of these tools.

```

    url: Option<NonNull<c_char>>,
    out: Option<NonNull<c_char>>,
    out_len: usize,
) -> BmResult {
    let Some(out) = out else {
        //...
    };

    // Safety: caller ensured `out` points to at least `out_len` bytes (2.)
    unsafe { slice::from_raw_parts_mut(out.as_ptr(), out_len) };
}

```

Note how we number safety invariants and force every inline safety comment to either:

- delegate its local invariant to the surrounding function’s safety comment, in which case it must reference a numbered invariant; or
- explain exhaustively why the code itself upholds the local invariant.

This way we make sure that all invariants are either upheld by the function itself or correctly documented as a responsibility of the caller.

At the moment, the numbered-invariant convention is checked by discipline and PR review. Clippy can require `# Safety` documentation and comments on unsafe blocks, but it cannot verify that a local justification actually upholds an invariant. There are proposals for tools that could make this kind of traceability machine-checkable in the future.⁷

Mind the FFI tax

Every exposed function is API surface you’ll maintain forever, an unsafe contract to keep correct, and a per-call cost the compiler can’t optimize away. Cross-language LTO can inline across the boundary, at the cost of a real setup burden. The cheapest FFI function is the one you didn’t expose. Prefer coarse operations, such as `bm_thing_update_with(...)`, over one setter per field, and treat the boundary as a small set of verbs, not a mirror of your internal struct.

Head to the exercise

You’ll update your solution to exercise 1.6 with the two rules from this section.

Exercise

The exercise for this section is located in [01_intro/07_high_quality_ffi_apis](#) \newpage{=latex}

⁷There are a couple of related proposals, all in the “pre-RFC” stage. The most interesting is the [safety-tags pre-RFC](#).

Chapter 2

Chapter 2: Intermediate

From now on we will be porting a “real” C application: the `bm` bookmark manager CLI. We will learn how to approach real-world codebases, structure our approach, and debug our code when it breaks.

In this chapter we’ll pick our first module to rewrite, learn how C and Rust allocators (don’t) mix, debug across the language boundary with `lldb/gdb`, harden our unsafe code with `Clippy` lints, replace C-isms with Rust-isms, catch undefined behavior with `Miri`, and benchmark the port against the C baseline.

Exercises

The exercises for this section are located in `exercises/02_intermediate`

Exercise

The exercise for this section is located in `02_intermediate/00_README` \newpage{=latex}

2.1 How to rewrite a module

In the previous chapter you learned the FFI building blocks: `extern` blocks, generated bindings, FFI-safe types, and how to design a boundary that acts as a firewall. Now we’ll put them to work on a real codebase. Meet `bm`, a small bookmark manager CLI written in C. Over the rest of this course we will migrate it to Rust, one module at a time.

Here is an overview of the C source code:

```
exercises/_bm/src/  
├─ bookmark.c    the Bookmark type  
└─ bookmark.h
```

```

├── cli.c           entry point, argument parsing
├── index.c         in-memory bookmark index
├── index.h
├── internal.h      definitions shared by index.c and storage.c
├── normalize.c     URL and tag normalization
├── normalize.h
├── result.h        shared result codes and error-string helper
├── storage.c       on-disk persistence of .bm files
├── storage.h
├── tag.c           tag parsing and matching
├── tag.h
├── util.c          string helpers
└── util.h

```

Picking the first module

There is no universally best place to start. A leaf module is often a good choice because it has few dependencies. The program's entry point can also work if it mostly calls other modules through simple interfaces. Sometimes the best boundary does not match an existing source file at all, and it is worth extracting a small module before porting it.

The right choice depends on the codebase, but a few guidelines help:

- Prefer a small, clearly defined interface.
- Minimize the number of C functions and data structures the Rust code must use.
- Avoid shared global state and complicated ownership rules in the first port.
- Choose code with useful tests, so you can compare behavior before and after.
- Keep the change small enough to review and, if necessary, revert.

```

cli      → bookmark, index, normalize, result, tag, util
index    → bookmark, internal, normalize, storage, tag, util
storage  → bookmark, index, internal, normalize, util
tag      → normalize, util
bookmark → util
normalize → (no project dependencies)
util     → (no project dependencies)

```

Two things are worth noting. `normalize` and `util` are the only real leaves: every other module pulls in at least one of its siblings. And `index` and `storage` include each other, so this is not the clean tree the file names suggest. Tangles like that are normal in code that has been maintained for a while, and they are exactly what you want to know before choosing where to start.

For `bm`, we chose to start with `normalize.c`. It exposes two functions: one normalizes URLs and the other normalizes tags. Both write into buffers supplied by the caller, so no memory changes ownership at the FFI boundary. Why not `util.c` or `bookmark.c`? Both seem suitable too. `util.c` is also small, but it is mostly thin wrappers around C string and allocation functions. `bookmark.c` contains more substantial logic, but it allocates values that must later be freed across the language boundary. We cover that problem in the next section.

Preserve the existing contract

When possible, an incremental migration replaces the module's *object file* without changing the interface used by the remaining C code. The existing C header (`normalize.h` in this example) describes the ABI our Rust implementation must initially satisfy. We re-implement every function the header declares in Rust and export it under the same symbol name using `#[unsafe(no_mangle)]` and `extern "C"`:

```
#[unsafe(no_mangle)]
pub unsafe extern "C" fn bm_normalize_url(
    url: Option<NonNull<c_char>>,
    out: Option<NonNull<c_char>>,
    out_len: usize,
) -> BmResult {
    // ...
}
```

The rest of the C code doesn't know, and doesn't need to know, that the implementation behind the symbol changed. The linker resolves the same symbol names as before; it just finds them in our Rust static library instead of the old object file.

That is a migration constraint, not necessarily the interface we want in the long term. Some C headers expose globals, macros, shared data structures, or ownership assumptions that cannot be reproduced cleanly in Rust. In those cases, we can introduce a small compatibility layer and improve the interface separately.

After that, all you have to do is link against the replacement library written in Rust instead of the original one written in C.

```
before: cli.o, index.o, normalize.o (C), ...
after:  cli.o, index.o, libnormalize.a (Rust), ...
```

Structuring the Rust side

Inside the crate we keep two layers, following the firewall pattern from section 1.6:

1. a thin `extern "C"` surface that validates raw pointers and converts C types at the boundary, and
2. a safe, idiomatic core that does the actual work with `&str`, `String`, and `Result`.

The safe core is where all new logic lives, and it's plain Rust: unit-testable with `cargo test`, no `unsafe` in sight. The FFI layer should stay boring.

Verifying behavior parity

A rewrite is only done when the observable behavior is unchanged. We have two safety nets:

- the module's existing C test suite, which now links against our Rust implementation and must keep passing, and
- new Rust unit tests against the safe core, which will outlive the C tests.

Tips

If the linker complains about an undefined symbol, inspect the symbols your Rust static library exports. The command depends on your toolchain and platform:

- Unix-like systems: `nm target/debug/lib<crate>.a`
- LLVM toolchains: `llvm-nm target/debug/lib<crate>.a`
- Windows with MSVC: `dumpbin /symbols target\debug\<crate>.lib`

A missing `#[unsafe(no_mangle)]` is a common cause.

While both implementations still exist, you can also run the same inputs through the C and Rust versions and compare their return values and output buffers. This is called differential testing. It catches small differences in behavior that the existing tests may not cover.

Exercise

The exercise for this section is located in [02_intermediate/01_rewrite_a_module](#)
`\newpage{=latex}`

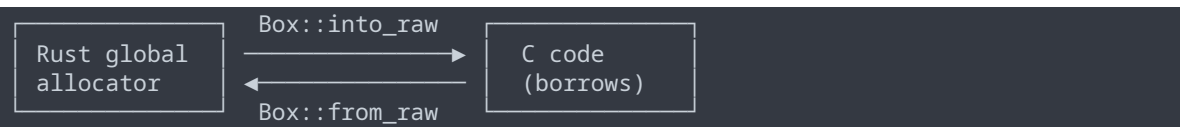
2.2 Allocation, allocators, and mixing them

In the previous section you saw how to replace a whole C module with Rust while keeping the interface identical. But `bm` passes more than integers and borrowed strings across the boundary: it passes *ownership*. `bookmark_new` allocates a `Bookmark` that someone else will free later. Who frees it, and with what? This is where mixed-language codebases hide some of their nastiest bugs and most difficult design challenges.

The golden rule

Every allocation must be released through its corresponding deallocation API.

That's the whole rule. C's `malloc/free`¹ and Rust's global allocator are two independent book-keeping systems². Handing a Rust-allocated pointer to C's `free` (or a `malloc`'d pointer to `Box::from_raw`) is undefined behavior, even if it happens to “work” on your machine because both forward to the same underlying `malloc` today. They might stop doing so tomorrow, or not do so at all on a different architecture.



¹`malloc/free` are standard C allocation functions, but nothing prevents you from using other allocation functions or even rolling your own. For the sake of simplicity, we will always refer to `malloc/free` in this section, but read it as `malloc/free` and all other custom (de)allocation functions.

²Currently, Rust's default global allocator is unspecified, but on many platforms it *is* the system `malloc`. Libraries such as `cdylibs` and `staticlibs`, however, are guaranteed to use the System allocator by default. Beware though: C's `malloc/free` API and Rust's global-allocation API are not interchangeable, even when both happen to use the same underlying system allocator.

Crossing the boundary in practice

There are several workable strategies:

1. **Use C allocation APIs from Rust.** Rust can call a C constructor such as `bookmark_new` and wrap the returned pointer in an owning Rust type. Its `Drop` implementation releases the allocation by calling the matching C function, `bookmark_free`. Keep allocation and deallocation within the same C API; export the functions through FFI to Rust.

Here is the complete shape of that wrapper. `OwnedCBookmark` owns the C allocation, so callers cannot accidentally pass it to Rust's allocator. A null pointer from C becomes `None`; dropping a successfully created wrapper always calls back into C to free it.

```
use std::ffi::CStr;
use std::ptr::{self, NonNull};

#[repr(C)]
// An opaque type, C owns the layout and allocation.
struct CBookmark {
    _data: (),
    _marker: core::marker::PhantomData<(*mut u8, core::marker::PhantomPinned)>,
}

// FFI (de)allocation functions
unsafe extern "C" {
    fn bookmark_new(
        url: *const std::ffi::c_char,
        tags: *const *const std::ffi::c_char,
        n_tags: usize,
    ) -> *mut CBookmark;
    fn bookmark_free(bookmark: *mut CBookmark);
}

// The Rust wrapper
struct OwnedCBookmark(NonNull<CBookmark>);

impl OwnedCBookmark {
    fn new(url: &CStr) -> Option<Self> {
        // SAFETY: `url` is NUL-terminated. This example passes no tags,
        // matching the C function's documented null-and-zero contract.
        let bookmark = unsafe { bookmark_new(url.as_ptr(), ptr::null(), 0) };
        NonNull::new(bookmark).map(Self)
    }
}

impl Drop for OwnedCBookmark {
    fn drop(&mut self) {
        // SAFETY: this pointer came from `bookmark_new` and this wrapper is
        // its unique owner, so `bookmark_free` is called exactly once.
        unsafe { bookmark_free(self.0.as_ptr()) };
    }
}
```

2. **Use Rust allocators from C.** A Rust type that crosses the boundary will need a matching

`_new/_free` pair exported through FFI. `_new` in Rust is paired with `_free` in Rust; C only ever holds the pointer and calls the Rust functions to allocate or free an object of that type. The basic pattern for this is to convert a `Box` to and from a raw pointer with `into_raw` and `from_raw`.³

```
let foo = Box::new(42);
let ptr = Box::into_raw(foo);
// This will convert the raw pointer back into a Box and when dropped free the memory:
let _foo = unsafe { Box::from_raw(ptr) };
```

And a full FFI split would look something like this:

```
use std::ptr::NonNull;

pub struct RustType {
    foo: i32,
    bar: i32,
}

#[unsafe(no_mangle)]
pub extern "C" fn rust_type_new(foo: i32, bar: i32) -> *mut RustType {
    Box::into_raw(Box::new(RustType { foo, bar }))
}

#[unsafe(no_mangle)]
pub extern "C" fn rust_type_free(b: Option<NonNull<RustType>>) {
    // To match C's free(NULL) semantics, return immediately when b is null.
    let Some(b) = b else {
        return;
    };

    // SAFETY: `b` was created by `rust_type_new` via Box::into_raw and is
    // not used again after this call (documented in bookmark.h).
    drop(unsafe { Box::from_raw(b.as_ptr()) });
}
```

3. **Transfer ownership of a C allocation across the boundary.** Use this when one language calls a C allocation API, transfers ownership of the result to the other language, and the receiving side later calls the matching C deallocation API. For example, Rust can allocate a NUL-terminated string with `libc::malloc` and pass it to C code that retains it and later calls `free`.

The reverse direction is valid too: C can transfer ownership of a value it allocated, for example with `malloc`, to Rust, and Rust can later call the matching deallocation function through FFI. In both cases, the allocation and deallocation calls originate on opposite sides of the FFI boundary. The layout and ownership contract must exactly match what the receiving API expects.

4. **Use a `malloc/free` adapter as Rust's global allocator.** On Linux, Rust's default global allocator typically already uses the system allocator, so this is usually unnecessary. It can

³There are some other useful structs, besides `Box`, that come with their own `from_raw` and `into_raw` functions, like `Vec` or `CString`. The same pattern as for `Box` applies here: allocate in Rust → convert into raw pointer and transfer ownership to C → use the pointer in C → free the pointer by transferring ownership back to Rust.

still be useful when Rust must use a specific C allocation API. Implementing the adapter is non-trivial: it's inherently unsafe, and you need to carefully uphold all the safety guarantees required by the `GlobalAlloc` trait. Do not pass Rust-owned values to C for it to free. Even with a `malloc/free` global allocator, C cannot run Rust destructors; freeing a value containing a `String`, `Vec`, or other owned resource leaks what that value owns.

Avoid if possible. An allocation adapter is trickier than it seems. The code is small, but getting its allocation contract right requires a lot of implicit knowledge. Only use one when you have a specific interoperability need.

5. **Use the Allocator API.** As of this writing, the API is still experimental and only available on nightly Rust. It functions similarly to `GlobalAlloc`: you have to implement your own allocator, but unlike the global one, it is a lot more flexible and can be used on a per-variable basis. The new API adds an `Allocator` trait and `Allocator` type parameters to many allocation-owning types, such as `Box` or `Vec`.

```
pub struct Box<T, A = Global>()
// Global is the global allocator implementing the Allocator trait
where
    A: Allocator,
    T: ?Sized;
```

Additionally, it adds a few new functions where you can pass an allocator, for example `new_in`:

```
#![feature(allocator_api)]

use std::alloc::System;
let foo = Box::new_in(1, System);

let custom_alloc = SomeCustomAlloc::new();
let bar = Box::new_in(2, custom_alloc);
```

`foo` and `bar` now use different allocators.

Avoid if possible. The same warning as for `GlobalAlloc` applies here: implementing an allocator is a non-trivial undertaking and is only worth it if you have a specific use case.

This API is primarily useful for arena or bump allocators, where many related values are allocated from the same memory region and released together. Another useful case is selectively integrating parts of your program with a different allocation system while still using standard owning types such as `Box` and `Vec`.

Spotting allocator mismatches

Tools like Valgrind give every allocation a birth certificate: run the test binary under valgrind - -leak-check=full and it will tell you not just that a block leaked, but which call stack allocated it.

Valgrind has no Apple Silicon support, so on an M-series Mac reach for leaks, for AddressSanitizer via `-Zsanitizer=address`, or, for the Rust side of the boundary, for Miri, which we cover in a later section.

Head to the exercise

You'll port `bm`'s `Bookmark` type, including its allocation and deallocation functions, to Rust without leaking or double-freeing a single byte.

You'll practice both ownership directions from this section in exercises `/02_intermediate/02_allocators`: C holding a Rust allocation, and Rust holding a C allocation.

Exercise

The exercise for this section is located in `02_intermediate/02_allocators` \newpage{=latex}

2.3 Using lldb/gdb

At some point in a migration, you'll hit a crash at the FFI boundary with little to go on. A C caller passes a string without the NUL terminator the API requires, or Rust hands C a pointer whose lifetime has ended. The process takes a `SIGSEGV` and dies before Rust can give you a tidy panic message or backtrace.

`gdb` and `lldb` can show the mixed C/Rust stack. Set a breakpoint in Rust, step into C, and inspect both sides of the boundary in one session.

Build with debug information

Build both sides of the boundary with debug information. Cargo's dev profile already sets `debug = true`; for a release build, add this to `Cargo.toml`:

```
[profile.release]
debug = true
```

If `build.rs` compiles the C code with `cc`, pass `-g` there too:

```
cc::Build::new()
    .file("c_src/shout.c")
    .include("c_src")
    .flag_if_supported("-g")
    .compile("shout");
```

Forget it and you get Rust frames with line numbers and C frames without: precisely the half of the stack you opened the debugger for.

Start the debugger

Use `rust-gdb` or `rust-lldb` rather than plain `gdb` and `lldb`. They're thin wrappers that load Rust's pretty-printers, which is what makes a `String`, `Vec`, or `Option` readable instead of a pile of pointers and capacities.

On macOS, that's usually `rust-lldb`:

```
rust-lldb target/debug/bm_lldb_gdb
```

On Linux, `rust-gdb`:

```
rust-gdb target/debug/bm_lldb_gdb
```

Program arguments go after `--` for `rust-lldb` and after `--args` for `rust-gdb`.

If you prefer a graphical frontend, `rust-gdbgui` starts the browser-based `gdbgui` with the same pretty-printers loaded. Install `gdbgui` separately.

```
rust-gdbgui target/debug/bm_lldb_gdb
```

Walk across the boundary

If you already know one debugger and need the equivalent command in the other, keep the official [GDB to LLDB command map](#) open. It covers both directions in one place.

Say you're investigating `bm_shout`, the C helper called by the Rust exercise. Set a breakpoint on it, run the program, and look at the stack when execution stops (output abbreviated):

```
(lldb) b bm_shout
Breakpoint 1: where = bm`bm_shout at shout.c:6
(lldb) run
Process 51234 stopped
* thread #1, stop reason = breakpoint 1.1
  * frame #0: bm`bm_shout at shout.c:6
    frame #1: bm`bm_lldb_gdb::shout at lib.rs:17
    frame #2: bm`bm_lldb_gdb::demo at lib.rs:30
    frame #3: bm`main at main.rs:2
```

C frames and Rust frames are interleaved, and the numbering tells you which way to walk: frame #0 is where execution stopped, and every higher number is one of its callers. *up* moves toward callers, *down* back toward the stop. So the code that handed over the bad value is always *up*, whichever language crashed. If Rust crashed, *up* gets you to the C caller and the arguments it passed. If C crashed, *up* gets you to the Rust frame that supplied them.

From there the usual commands work in both debuggers: *s* steps into a call, *n* steps over one, *finish* runs the current frame to completion, *bt* prints the stack, and *p* prints a value.

For a `char *out`, inspect the bytes directly: `x/s out` in `gdb`, and either `x/s out` or `memory read --format c-string out` in `lldb` (the format name is `c-string`, with the hyphen; `cstring` is rejected). For a raw buffer, use `x/16xb out` in `gdb` or `memory read --size 1 --count 16 --format x out` in `lldb`. This quickly tells you whether the pointer, length, and terminator agree with the FFI contract.

When the process dies outright, you don't need a breakpoint at all. Run it, let it take the signal, and print the stack:

```
(lldb) run
Process 51234 stopped
* thread #1, stop reason = EXC_BAD_ACCESS (code=1, address=0x0)
(lldb) bt
```

The frame you land in is where the bad pointer was dereferenced, which is not necessarily where the bug is. Walk up until the values stop making sense.

A note on Rust symbol names

Everything above assumes your debugger can read the names in that stack. Since Rust 1.97, `rustc` uses the v0 symbol-mangling scheme by default. Nightly has done so since November 2025, so the toolchain this book pins already emits v0 symbols.

Symbol mangling is how the compiler turns a function into a linker-safe name. For a C function, that's usually just the function name. Rust also has to distinguish generic instantiations: `Vec<u8>::push` and `Vec<String>::push` are different generated functions and therefore need different symbols.

The old scheme used a hash for much of that distinction. It worked, but a backtrace through generic code often left you with a readable function name and an opaque suffix. The v0 scheme preserves the concrete generic arguments in a form tools can decode, so when you're debugging or profiling a monomorphized call chain, the backtrace can show which instantiation actually ran rather than a hash.

This doesn't affect functions exported to C with `#[unsafe(no_mangle)]`, but it does require every tool that reads Rust symbols to understand v0. Recent `gdb`, `lldb`, and Rust tooling do. Older debuggers, profilers, or crash-symbolication pipelines may show raw names such as `_RNVNtNtCs...` instead. Check your whole debugging path before upgrading a pinned CI or production toolchain.

A quick test is enough: write a small generic function that panics, run it with `RUST_BACKTRACE=1`, and read the backtrace.

See the [stabilization PR](#) and the [v0 symbol-format documentation](#) for details.

Head to the exercise

In this exercise a small C helper and its Rust binding disagree about something that only shows up once you call across the boundary. Build it with `cargo build -p bm_lldb_gdb`, then debug `target/debug/bm_lldb_gdb`. No `println!` allowed.

Exercise

The exercise for this section is located in [02_intermediate/03_lldb_gdb](#) \newpage{=latex}

2.4 Clippy lints for unsafe Rust and static verification

In the previous section we learned how to debug an application, but debugging only gets you so far: it starts from a crash you already have, and works backward to the root cause.

Lints, on the other hand, help prevent certain issues in the first place. They catch a narrower set of problems and suspicious patterns before the code runs. Running them can be fully automated, which helps enforce coding standards across an entire team.

They are especially useful in unsafe code. They can require each unsafe operation to be explicit, flag signatures that don't state the caller's obligations, and surface undocumented assumptions. They can't prove that a pointer is valid or that a C caller is honest, but they can make unchecked assumptions visible for review.

Built-in lints

You already have several useful lints enabled by default.

`improper_ctypes` and `improper_ctypes_definitions` are `rustc` lints, warn-by-default, and you've encountered them already: they're what complained about the non-FFI-safe types back in Chapter 1. `improper_ctypes` covers the types you declare in an `extern` block; `improper_ctypes_definitions` covers the ones you define with `extern "C"`. Between them they catch most type-level mistakes long before a linker gets involved.

`clippy::not_unsafe_ptr_arg_deref` is a correctness lint, so Clippy already marks it as `deny` rather than `warn`. It fires when a public safe function dereferences a raw-pointer argument even though its signature places no requirements on the caller. Either make the function unsafe and document its pointer contract, or keep it safe and accept a type that can be dereferenced safely. Don't silence the lint.

`clippy::missing_safety_doc` flags a public `unsafe fn` whose docs have no `# Safety` section. Set `check-private-items = true` in `clippy.toml` to apply it to private items as well. This is worth doing in an FFI crate, where plenty of the interesting unsafe code isn't public.

More lints to turn on

You can configure a lint in code:

```
#![warn(clippy::undocumented_unsafe_blocks)]
```

Or you can use lint tables in `Cargo.toml`. We'll use `Cargo.toml` here to keep this crate-wide policy in one place:⁴

```
[lints.rust]
unsafe_op_in_unsafe_fn = "deny"

[lints.clippy]
undocumented_unsafe_blocks = "warn"
```

⁴The complete list of Clippy lints is available at <https://rust-lang.github.io/rust-clippy/master/index.html>.

```
multiple_unsafe_ops_per_block = "warn"
unnecessary_safety_doc = "warn"
ptr_as_ptr = "warn"
cast_ptr_alignment = "warn"
transmute_ptr_to_ptr = "warn"
```

This section focuses on Clippy, but `unsafe_op_in_unsafe_fn` is a compiler lint, not a Clippy lint. It's worth mentioning here because it makes unsafe operations explicit, which is the foundation for the Clippy lints that follow. An `unsafe fn` means its caller has extra obligations. It doesn't mean every unsafe operation in the body is automatically justified. With this lint enabled, each dereference, FFI call, or other unsafe operation needs its own `unsafe {}` block. It's allow-by-default in edition 2021 and warn-by-default in edition 2024, so it's worth denying explicitly either way.

Two lints take that idea a step further. `clippy::undocumented_unsafe_blocks` asks for a `// SAFETY:` comment before each unsafe block. `clippy::multiple_unsafe_ops_per_block` encourages each block to contain a single unsafe operation, keeping its safety argument focused. Together they turn a large, vague unsafe region into small claims that can be checked one by one.

`clippy::unnecessary_safety_doc` catches the opposite documentation mistake: a `# Safety` section on a public safe function or trait. A safe API should not have safety preconditions for its callers or implementors to uphold. This is a restriction lint and allow-by-default, so enable it explicitly.

The pointer lints earn their keep when you're porting C habits mechanically. Pointer casts and pointer transmutes often compile without complaint, even when they introduce a stricter alignment requirement or hide a more direct API. A warning doesn't necessarily mean that the code is wrong, but it's a good reason to double-check what the cast is claiming.

When the lint is wrong

Porting from C can also produce the opposite problem. Some C naming conventions conflict directly with rustc's style lints: a mirrored `bm_db_t` typedef triggers `non_camel_case_types`, an extern static trips `non_upper_case_globals`, and a function or variable that keeps its original name trips `non_snake_case`. The group covering all three is `nonstandard_style`.

Sometimes the name genuinely does have to match, and then an `allow` is the right solution. Scope it as tightly as you can:

```
#[allow(non_camel_case_types)]
pub struct bm_db_t { /* ... */ }
```

A crate-root `#![allow(non_snake_case)]` also silences names you genuinely got wrong. Prefer applying allows to individual items instead of whole modules or entire crates. Generated code you don't control is the exception that can justify a module-wide allow. Put it on the module containing the generated bindings rather than on the entire crate.

You can often avoid an `allow` for functions exposed to C. `no_mangle` uses the Rust function's name for the exported symbol, but `export_name` lets you choose a different name for the C API:

```
#[unsafe(export_name = "BMNormalizeURL")]
pub extern "C" fn bm_normalize_url() {
    // ...
}
```

Note: cheadergen recognizes `export_name`, so it emits `BMNormalizeURL` in the C header while the Rust function keeps its idiomatic name.

Share lint settings across a workspace

The configuration above applies to one crate. To use it across a workspace, put the tables in the root `Cargo.toml` instead. They've been available since Rust 1.74:

```
# Cargo.toml at the workspace root
[workspace.lints.rust]
unsafe_op_in_unsafe_fn = "deny"

[workspace.lints.clippy]
undocumented_unsafe_blocks = "warn"
multiple_unsafe_ops_per_block = "warn"
```

Each member crate then opts in with:

```
[lints]
workspace = true
```

// SAFETY: comments are part of the code

Consider a pointer that arrives across the FFI boundary:

```
use std::ffi::CStr;
let raw = c"https://example.com".as_ptr();
// SAFETY: `raw` is non-null (checked above) and points to a valid
// NUL-terminated byte sequence, as required by `bm_normalize_url`'s contract.
let raw = unsafe { CStr::from_ptr(raw) };
```

The null check happens in this function; the valid NUL-terminated sequence is part of the caller's documented contract. A reviewer can check both claims without re-deriving them from the surrounding code.

Compare that with this:

```
use std::ffi::CStr;
let raw = c"https://example.com".as_ptr();
// SAFETY: this is fine
let raw = unsafe { CStr::from_ptr(raw) };
```

That comment carries no information. The next reader still has to establish whether `raw` is non-null, initialized, and terminated. Write the reasoning down.

It's the same discipline as the `# Safety` documentation from Chapter 1. The function's `# Safety` section describes what its caller must guarantee. The local `// SAFETY:` comment describes why one operation can rely on those guarantees at this exact point in the implementation.

Lints are a starting point, not a proof

Run the lints with:

```
cargo clippy --workspace --all-targets
```

`--all-targets` matters here. Without it, `cargo clippy` skips tests, and in these exercises the tests are where most of the FFI calls live.

`cargo clippy --fix` can apply some mechanical fixes. It wants a clean Git tree and refuses to run on a dirty one unless you pass `--allow-dirty`. Review the diff: replacing a pointer cast is easy; deciding whether the pointer is correctly aligned, valid, and live is the more difficult work.

Other mistakes won't trigger a lint at all. No compiler warning can tell you that a C caller lies about a buffer length, or that two libraries expect different allocators. Keep writing tests, use a debugger when they fail, and treat every FFI contract as something to validate at the boundary.

Head to the exercise

You'll work through a small FFI module with the full lint set already switched on. Some warnings are mechanical, others point to a real bug. The point isn't to make `cargo clippy` quiet. It's to understand and document every unsafe operation that remains.

Exercise

The exercise for this section is located in [02_intermediate/04_clippy_lints](#) \newpage{=latex}

2.5 Replacing C-isms with Rust-isms

One of the hardest things when translating one language into another is *idioms*: common patterns, practices, and expressions. These are shaped by the affordances of the language and by the authors' cultures, norms, and backgrounds.

Translating between programming languages runs into the same problem as translating between human ones. You can do a *literal* translation of a function or module, but it won't be right: it won't sit well with the code around it, it will cause confusion, and it will destroy confidence in the code that the rewrite was supposed to build.

The good news is that these translations follow fairly regular patterns and are easy to wrap your head around. This section walks through several of them, starting with the easy ones:

Out parameters

C functions make frequent use of so-called “out parameters”, pointers that the function will write its outputs into. In a lot of situations this is great, because the caller can allocate the output however it sees fit. But that flexibility brings a lot of potential for ambiguity:

```
int add(int a, int b, int *out);
```

At a glance: what does this function return, and how does it return it? `out` is a mutable pointer, so add could write the output there, but it also returns `int`, so maybe that’s where the output goes?

We have no way of knowing for sure. Here is the equivalent Rust function:

```
fn add(a: i32, b: i32) -> Option<i32>;
```

It becomes clearer that `add` is some kind of “checked” addition function that returns an output only sometimes, and otherwise fails.

This is fundamentally a limitation of what’s possible with C’s type system. A return code plus an out parameter is what you will often find in the wild, and nothing in the signature tells you which is which. In Rust’s more expressive type system we can write `Option<T>` to signal the intent of our code to callers from the signature alone.

Error codes

Likewise, in C code you will often see the following:

```
#define ERR_OK 0
#define ERR_FOO 1
#define ERR_BAR 2

int do_something();
```

You may also see it written as a typedef enum, which is common in more modern code and is what `bm` does.

Much like with out parameters, correct usage relies on conventions, good documentation, and proper code review. The compiler will not catch you accidentally using the wrong constant: C converts these to and from `int` freely.

In Rust we would express this using an enum (actually a combination of enums):

```
enum Error {
    Foo,
    Bar,
}

fn do_something() -> Result<(), Error>;
```

Here is a neat trick: an enum whose variants carry no fields (a so-called *field-less* enum) is nothing but an integer tag, and you can cast it to an integer with `as`:

```
enum Error {
    Foo,          // variants are numbered starting at zero
    Bar = 45,     // explicit tag
}

println!("{}", Error::Foo as usize); // prints 0
println!("{}", Error::Bar as usize); // print 45
```

If you want to make sure tags match your old C version exactly (for interop purposes, for example), you might do this:

```
#[repr(C)] // use the target's C ABI representation for enums
enum Error {
    Foo = 0,
    Bar = 1,
}

fn do_something() -> Result<(), Error>;
```

`#[repr(C)]` gives `Error` the size and signedness a C compiler would pick for the equivalent typedef enum on the same target, so the two agree on the values they exchange. Reach for `#[repr(i32)]` when you want to pin the width yourself, which is what the `#define` version above needs, since it passes plain ints around. Don't reach for `#[repr(u32)]`: C's int is signed, and the difference surfaces the first time someone adds a negative error code.

Strings

Strings deserve extra care. The obvious translation would be `char * => &str` (or `String`):

```
void greet(const char *name);
```

```
fn greet(name: &str);
```

But C and Rust don't fully agree on what a string *is*. A C string is a pointer to some bytes that end at the first NUL byte. A `&str` carries its length with it, has no terminator, and must be valid UTF-8. Handing a `&str` to C directly, or casting a `char*` to a `&str` in Rust, will cause immediate problems. C will read past the end of a `&str` looking for a NUL that isn't there, and a `char*` isn't necessarily valid UTF-8 at all.

The correct bridge is `CStr` and `CString`: `CStr::from_ptr(p).to_str()` checks the bytes and gives you a `Result<&str, Utf8Error>`; `CString::new(s)` appends the terminator and refuses anything with an interior NUL.⁵

⁵Since Rust 1.77 you can write `c"hello"`, which gets you a `&'static CStr` directly.

Bit flags

Same story as error codes, just with bits:

```
#define FLAG_READ  (1 << 0)
#define FLAG_WRITE (1 << 1)
#define FLAG_EXEC  (1 << 2)

int open_file(const char *path, int flags);
```

flags is an int, so nothing stops you from passing ERR_F00 or 42. The bitflags crate generates a newtype around the integer with the named constants, the |, &, and ! operators, and methods like contains, insert, and remove:

```
use bitflags::bitflags;

bitflags! {
    #[repr(transparent)]
    #[derive(Clone, Copy, Debug, PartialEq, Eq)]
    struct Flags: u32 {
        const READ  = 1 << 0;
        const WRITE = 1 << 1;
        const EXEC  = 1 << 2;
    }
}

fn open_file(path: &Path, flags: Flags);
```

Now the signature says what it accepts. For interop, the same tricks as before apply: #[repr(transparent)] gives Flags the exact layout of the u32 it wraps, bits() gets the raw integer back out, and from_bits(x) returns None if any bit you didn't define is set. If C hands you values with bits you don't know about (newer headers, reserved bits), use from_bits_retain, which keeps them as they are.

Loops and iterators

Here is the matches function from bm's index:

```
for (size_t i = 0; i < b->n_tags; i++) {
    if (strstr(b->tags[i], query))
        return 1;
}
return 0;
```

The literal translation is:

```
struct Bookmark { tags: Vec<String> }
fn matches(b: &Bookmark, query: &str) -> bool {
    for i in 0..b.tags.len() { if b.tags[i].contains(query) { return true; } }
    false
}
```

It works, but every `b.tags[i]` is a bounds check the optimizer may or may not be able to prove away, and Clippy will nag you about it (`needless_range_loop`). Iterate the collection itself instead: `for tag in &b.tags`, with `.enumerate()` if you actually need the index and `.iter_mut()` if you need to change elements.

From there it is a short step to the combinators, and the loop above is just any and looks like this:

```
struct Bookmark { tags: Vec<String> }
fn matches(b: &Bookmark, query: &str) -> bool {
    b.tags.iter().any(|tag| tag.contains(query))
}
```

The other common shape is the pointer-bumping loop:

```
for (const char *p = s; *p; p++) {
    if (*p == ',')
        n++;
}
```

This one walks bytes until it hits the NUL terminator. In Rust the string knows its own length, so loops like these become `.bytes()` or `.chars()`.

The pointer-bumping loop would look like this in Rust:

```
let s = "rust,ffi,c";
let n = s.bytes().filter(|&b| b == b',').count();
```

A loop that searches for an element is `find` or `position`; one with a running total is `map(..).sum()`; one that fills a second array is `filter(..).map(..).collect()`. These are lazy and compile down to the same code as the hand-written loop, so there is no cost to the shorter form, and the name of the combinator tells the reader what the loop was for.

Exercise

The exercise for this section is located in [02_intermediate/05_idiomatic_rust](#) \newpage{=latex}

2.6 Dynamic analysis with Miri

In Chapter 1 we introduced the “firewall” rules that keep FFI interfaces sane, and in the previous section we looked at lint rules that help enforce them. Lints and compiler settings can’t observe behavior, though. They can only read your code as the compiler does, and infer from that.

We’ll now introduce another class of tooling to complement static analysis: dynamic analysis with Miri.

Miri is an interpreter for rustc’s MIR (mid-level intermediate representation). It runs your code, checking each operation against the Rust Abstract Machine. Miri is especially useful for unsafe Rust, since it catches incorrect initialization, use-after-free, and borrowing-rule violations: a good

fit for the kind of systems code we tend to translate from C to Rust. Do note that Miri requires a nightly compiler. This repository's `rust-toolchain.toml` already pins one with the `miri` component included, so the exercises work without any extra setup.

What Miri catches

When interpreting your Rust code, Miri tracks the initialization state and provenance of each allocation. It will flag reads outside the bounds of your allocation, before it's correctly initialized (unless you use `MaybeUninit`), or after it's freed. Miri also catches data races, including weak-memory reorderings, and the aliasing violations that complex unsafe code tends to produce, such as writing through a mutable reference while other code still holds references to the same allocation. It also catches memory leaks: if a test ends without freeing every allocation, for example through `Box::into_raw` or `CString::into_raw` without a matching `from_raw`, Miri will fail the test.

The downsides

Miri is by design an interpreter, meaning it can only ever see code paths that actually execute. Code paths never exercised by your test suite are invisible to Miri. This also means non-deterministic bugs need luck: a bug that only happens on a specific thread interleaving will remain flaky.⁶ Miri is also slow: expect a 3,000–7,000× slowdown in some cases.

Quick Miri cheat sheet

Using Miri is just like using Cargo, but with `miri` inserted into the commands. For example, to run unit tests: `cargo miri test -p <crate>`. You can also run binaries, examples, etc.

To pass flags to Miri itself, use the `MIRIFLAGS` environment variable.

The first `cargo miri` invocation builds a Miri-specific sysroot, which takes a minute or two. Run `cargo miri setup` once ahead of time to get that out of the way.

Quite often you will need to tweak your code and test cases to run under Miri at reasonable speed, or at all. For example, you may want to reduce test iterations when running under Miri or disable certain incompatible tests. For this, the `miri cfg` attribute exists. You can write `#[cfg(miri)]` on expressions to enable them *only* when executing under Miri. For incompatible tests you can write `#[cfg_attr(miri, ignore)]`.

Note that Miri (being an interpreter) also accepts any `--target` triple which is handy to test certain target features such as big-endian or 32-bit without running an actual emulator of some sort.

⁶The `-Zmiri-many-seeds` option runs several executions at once, which helps catch flakes more often. Miri also has the `-Zmiri-address-reuse-rate`, `-Zmiri-address-reuse-cross-thread-rate`, `-Zmiri-preemption-rate`, and `-Zmiri-compare-exchange-weak-failure-rate` options that help with catching threading bugs more often. But generally it's better to turn to tools like [loom](#) for proper concurrency testing.

Miri and C

Miri is a pure Rust interpreter and cannot run native libraries. The Miri team hand-wrote shims for common ones such as `libc`, which are built into Miri, but for the long tail of native libraries there is no support. Unfortunately, our mixed C-Rust codebase sits right in that long tail. So the question isn't "can Miri check my mixed codebase?" (it cannot), but "how much of it can I get Miri to check?"

The strategy we most often employ is "thin glue, thick core": pure-Rust core with `unsafe extern "C"` functions (the uncheckable part) being extremely simple adapters only. This means the vast majority of the code we actually care about is already Miri-checkable, and FFI functions can be validated using lints, reviews, and other tools if necessary.

The only complicating factor is Rust calling into C: our Rust code calling a not-yet-translated module, for example, or some native library we do not intend to port at all. For this we use hand-written shims (just like the Miri team did) that swap out the C functions for Rust stubs when running under Miri. Keep in mind that the stub doesn't need to implement the C code at all; it simply needs to return the right values: error codes, out-params, and buffers of a given length. One neat side effect is that "stubbing out a C function" forces you to confront the details of a function that we gloss over when we simply call it. That usually leads to a much better understanding of the code, which makes it a worthwhile exercise in itself.

You would usually have something like this:

```
#[cfg(miri)]
mod stubs {
    #[unsafe(no_mangle)]
    unsafe extern "C" fn SomeCFunction() {
        // ... some stub here ...
    }
}
```

In a regular build we would link against the `.so` library and get the `SomeCFunction` symbol that way. Under Miri we stub out the symbol, pointing it at a simple pure-Rust function. The stubs live in their own module because Rust won't let a definition and the `extern` declaration of `SomeCFunction` share a name.

One word of caution, though: Miri passing with Rust stubs does not mean your codebase with the full C functions is correct. These stubs, written correctly, can give you confidence that the final product will also be correct, but they are not a substitute for full-program tests. It is easy to accidentally test your stubs' behavior more than the actual C code's.

Future features

Miri has gained support for running code directly from native libraries: `MIRIFLAGS="-Zmiri-native-lib=path/to/libbm.so"` loads the `.so` and directly calls the symbols in it. This is a promising direction, but for now it is limited. It does not support Windows, supports only integer and pointer arguments and returns (no structs by value), and by design cannot check most Rust invariants: memory shared with C stops being tracked for init and provenance. On Linux there is even more experimental support for native tracing that attempts to trace which bytes C touched and keep tracking for all others, but none of it is ready for production use yet.

Head to the exercise

The exercise is in `exercises/02_intermediate/06_miri`. It contains a small Rust wrapper around `tag_normalize` from `tag.c`, a module we haven't ported yet. Its tests pass with a plain `cargo test` but fail under Miri.

Exercise

The exercise for this section is located in `02_intermediate/06_miri` \newpage{=latex}

2.7 Performance benchmarking

You have ported a C function to Rust. The tests pass and the output is identical. Great! But what about execution speed?

It is tempting to run the program once, look at the wall clock, and call that a benchmark. The problem is that one number tells you almost nothing. Your laptop may boost one run and throttle the next. The filesystem cache may already be warm, and your browser may wake up and steal a core.

A *benchmark* repeatedly measures a controlled workload. It can report *latency*, how long one operation takes, or *throughput*, how many operations finish in a given time. Our benchmark will measure the latency of one call to `bm_normalize_url`.

A *profile* answers a different question: where did a program spend CPU time, allocate memory, or perform I/O? We focus on benchmarking here because it tells us whether the C-to-Rust port regressed. Profiling appears briefly at the end, as a way to investigate an unexpected result.

Benchmarking tools

The main tool for running benchmarks is `cargo bench`. By itself it does nothing and requires a benchmark harness. At the time of writing, Rust's built-in benchmark harness is nightly-only: `#[bench]`, the `test` crate, and `test::Bencher` are all unstable.⁷ In stable Rust, you need a third-party harness such as `Criterion` or `Divan`. `Criterion` stores baselines and reports changes between runs. `Divan` provides a smaller, attribute-based harness and reports the fastest, slowest, median, and mean times. For simplicity, we will use `Divan`.

Add `Divan` to the crate that owns the Rust port. The exercise also uses the `cc` crate to compile the C baseline:

```
# Cargo.toml
[build-dependencies]
cc.workspace = true

[dev-dependencies]
divan = "0.1"
```

⁷See the [test library feature](#) in the Unstable Book for the current status of `#[bench]` and `test::Bencher`.

```
[[bench]]
name = "normalize"
harness = false
```

`harness = false` tells Cargo not to start its built-in test harness. The benchmark binary supplies its own main function and starts Divan instead:

```
fn main() {
    divan::main();
}
```

The build script compiles the original C implementation:

```
// Sample build.rs
fn main() {
    cc::Build::new()
        .file("c_src/normalize.c")
        .compile("bm_normalize_c");
}
```

This keeps both implementations in the same Cargo build. When `cargo bench` selects the optimized bench profile, `cc` reads Cargo's `OPT_LEVEL` and applies the corresponding optimization level to the C code too.⁸

If you build the C code manually instead of using `cc`, pass the appropriate `-O` flag to the C compiler yourself. Otherwise you may end up comparing optimized Rust with unoptimized C, which makes the benchmark meaningless.

Benchmark the same work

Now we need to compare the C implementation with the Rust port. A C/Rust comparison is only useful when both benchmarks include the same work.

Start from the C signature. The caller owns the output buffer, and nothing is allocated:

```
BmResult bm_normalize_url(const char *raw, char *out, size_t out_len);
```

Porting it gives you a choice, and this decides what you are able to measure:

```
// Safe and idiomatic. Not comparable: it allocates, and reaching it from
// C's `const char *` costs a UTF-8 check on the way in.
pub fn normalize_url(raw: &str) -> Result<String, InvalidUrl>;

// Comparable: the same contract as C, so the two benchmarks do the same work.
pub unsafe extern "C" fn bm_normalize_url_rust(
    raw: Option<NonNull<c_char>>,
    out: Option<NonNull<c_char>>,
    out_len: usize,
) -> BmResult;
```

⁸The `cc` crate reads the optimization level from Cargo's `OPT_LEVEL` environment variable. See the [cc::Build::opt_level](#) documentation.

If you benchmark the first one against C, the Rust number carries a validation and an allocation that the C number does not: you have measured two different APIs, not two implementations of the same operation.

This does not mean the safe API is wrong. The allocation may well be the right design. It does mean you need to state what the benchmark includes.

For a direct port comparison, take the second signature, and give both implementations the same input and equivalent reusable output buffers:

```
use std::ffi::c_char;
type BmResult = i32;
unsafe extern "C" {
    #[link_name = "bm_normalize_url"] // bind the C symbol under a second name
    fn bm_normalize_url_c(raw: *const c_char, out: *mut c_char, out_len: usize) -> BmResult;
}

#[divan::bench(sample_size = 4096)]
fn c(bencher: divan::Bench) {
    let raw = b"HTTPS://EXAMPLE.COM/Rust\0";
    let mut out = [0 as c_char; 2048]; // created once, so it is not timed

    bencher.bench_local(|| unsafe {
        let result = bm_normalize_url_c(
            divan::black_box(raw.as_ptr().cast()),
            divan::black_box(out.as_mut_ptr()),
            out.len(),
        );
        divan::black_box(result)
    });
}
```

The benchmark setup happens before `bench_local`, so creating the output buffer is not timed. The closure mutably captures that buffer and reuses it for every iteration. `bench_local` runs single-threaded.

These calls take only a few dozen nanoseconds. `sample_size = 4096` runs each benchmark that many times in every timing sample; Divan then divides the total time by 4096. This keeps each sample well above timer precision and makes the result less sensitive to an interruption during Divan's automatic calibration. Use the same sample size for both implementations; adjust it for slower or faster workloads.

black_box is not decorative. Without it, the compiler can notice that the result goes nowhere and optimize away some or all of the work. You would then be wowed by the speed of a function that did not run, making the whole benchmark meaningless.

Rule of thumb: Benchmark the semantics you want to compare, not the convenience API you happened to write first.

Check correctness first, then run both benchmarks, from the workspace root:

```
cargo test -p bm_benchmarking
cargo bench -p bm_benchmarking --bench normalize
```

You can pass a filter to Divan when you only want one implementation:

```
cargo bench -p bm_benchmarking --bench normalize -- rust
```

Divan invokes each function many times. It groups those invocations into samples and reports a per-iteration time:

- **median** is the middle sample and usually the most useful value for a quick comparison;
- **mean** is the average and moves more when one sample is unusually slow;
- **fastest** and **slowest** show the extremes and help expose noisy runs;
- **samples** and **iters** show how many measurements and function calls contributed to the result.

The fastest-to-slowest range shows only the extremes within one run. A single outlier can make it huge, so do not use it to decide whether two implementations differ.

Check for a regression

Suppose repeated runs put Rust around 220 ns and C around 200 ns. That is a 10% difference.

It is a *performance regression* only if the slowdown remains under the same workload and conditions. Run the benchmark several times on the same otherwise-idle machine and compare the medians. If the difference keeps changing direction, it is too small to distinguish from noise under those conditions. If Rust remains slower by roughly the same amount, investigate.

The instructions are deterministic; the conditions under which they run are not. Caches, frequency scaling, and whatever else the machine is doing all land in your measurement.

Treat a repeatable regression like any other bug:

1. Run the correctness tests first. A faster wrong port is not an optimization.
2. Record the benchmark command, input, machine, and baseline median.
3. Change one thing, then rerun the same benchmark under the same conditions.
4. If the regression remains, profile the slow path before changing more code.

Keep the benchmark after the fix. It is now the thing that will catch the regression if it comes back.

CI tip: A continuous-benchmarking tool such as [Bencher](#) can store results over time and fail a pull request when performance regresses.

A brief note on profiling

If a benchmark finds a repeatable regression, a profiler can show where the extra work happens.

[cargo-samply](#) builds an optimized binary with debug information and records CPU samples. See its documentation for installation and usage. Look for functions that occupy the widest frames in the profile. Those are where the sampled CPU time accumulated. Then record a profile for the C implementation, or for the version before your change, and compare it with the new profile. This shows which functions now take more CPU time and explains the slower benchmark.

[Hotpath](#) is useful when you want timing or allocation measurements for a small set of functions in an application. Its instrumentation is disabled unless you enable the corresponding Cargo features:

```
# Cargo.toml
[dependencies]
hotpath = "0.24"

[features]
hotpath = ["hotpath/hotpath"]
hotpath-alloc = ["hotpath/hotpath-alloc"]
```

Mark the functions you want to measure and the application's entry point:

```
#[hotpath::measure] // time every call to this function
fn normalize_url(raw: &str) -> String {
    // ...
}

#[hotpath::main] // install the reporter, print the table on exit
fn main() {
    // ...
}
```

```
cargo run --release --features='hotpath,hotpath-alloc'
```

Hotpath can also write JSON reports for CI. Its `hotpath-utils` command compares reports from the pull request and the base branch, including per-function timing and allocation differences.⁹ This is useful for watching a few representative application paths. Keep Divan for controlled microbenchmarks, and disable Hotpath while collecting the final Divan numbers because instrumentation changes the code being measured.

Head to the exercise

The exercise is in `exercises/02_intermediate/07_benchmarking`. It ships with the C benchmark. Replace the `bm_normalize_url_rust` stub and add the matching Rust benchmark, then run both and compare their median latencies.

The parity test compares return codes on every input, but compares the output buffers only when the call succeeded. The C version lowercases into the caller's buffer before it looks for a tab,

⁹The [Hotpath CI guide](#) shows how to produce the head and base JSON reports and compare them in GitHub Actions.

so it writes into out even on inputs it rejects. Your port validates first, so it doesn't. That is a difference differential testing is supposed to surface, and one worth keeping: the test pins it down rather than asking you to reproduce it.

Exercise

The exercise for this section is located in [02_intermediate/07_benchmarking](#) \newpage{=latex}

Mainmatter

mainmatter.com/rust
info@mainmatter.com